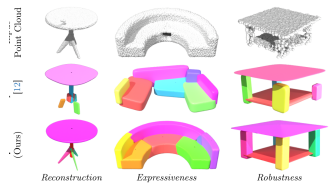
GEOMETRIC DEEP LEARNING / PAPER STUDY

SuperFlex

Deformable Superquadrics for Point Cloud Decomposition

Gabriel Tavernini, Elisabetta Fedele, Tiago Novello, Leonidas Guibas, Marc Pollefeys, Francis Engelmann

ECCV 2026



Figures reproduced from the paper, Fig. 1. Source: arXiv:2607.01015.

The actual task: interpret a 3D shape with a few parts

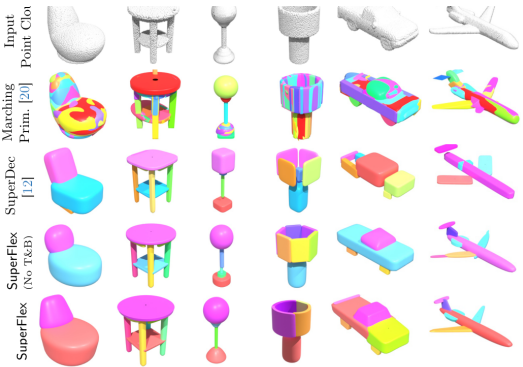
Input

A point cloud $\mathcal{P} = \{\mathbf{x}_i\}_{i=1}^{N}$, with $\mathbf{x}_i \in \mathbb{R}^3$.

Output

A compact **set of parametric solids** whose union reconstructs the object, with meaningful parts.

- **Fidelity**: closely match the target geometry.
- **Parsimony**: use few primitives.
- **Robustness**: cope with missing surfaces.



Paper Fig. 4: each colour represents one primitive.

Where does SuperFlex fit?

Optimization-based	Learning-based	SuperFlex (2026)
EMS (2022): probabilistic fitting, robust to outliers.	**SQ** (2019): learn primitive abstractions.	**Inherits** SuperDec's point-encoder + Transformer decoder.
Marching Primitives (2023): iterative SDF-driven fitting.	**CSA** (2021): joint cuboid assignment and prediction.	**Adds** expressive geometry, stronger losses, optional refinement and occlusion training.
	SuperDec (2025): set-prediction Transformer.	

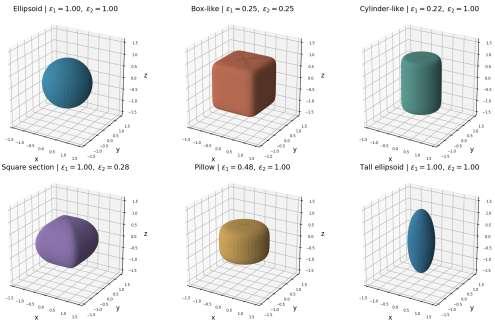
The mathematical building block: a superellipsoid

Canonical implicit surface

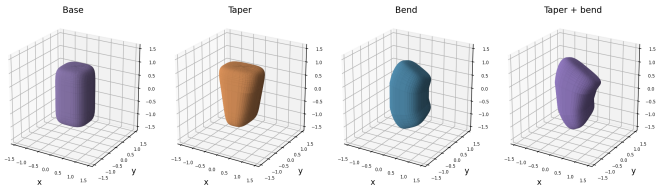
$$q_{\text{can}}(\mathbf{x}) = \left[\left| \frac{x}{s_x} \right|^{\frac{2}{\epsilon_2}} + \left| \frac{y}{s_y} \right|^{\frac{2}{\epsilon_2}} \right]^{\frac{\epsilon_2}{\epsilon_1}} + \left| \frac{z}{s_z} \right|^{\frac{2}{\epsilon_1}}$$

$$\mathcal{S} = \{\mathbf{x} : q_{\text{can}}(\mathbf{x}) = 1\}$$

- $\mathbf{s} = (s_x, s_y, s_z)$: axis scales.
- ϵ_1: meridional (vertical) profile.
- ϵ_2: horizontal cross-section.



Five canonical shape parameters are not enough



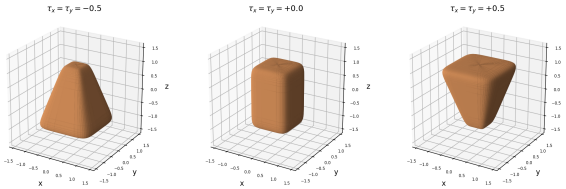
Failure of rigid primitives

Straight, symmetric parts are efficient to model—but curved and tapering parts need **more primitives**, even when a single semantic part would suffice.

- **Tapering**: changing cross-sectional width.
- **Bending**: non-straight central axis.

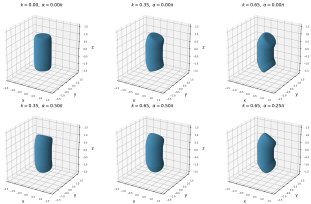
Deformation: what do τ, k, and α do?

Tapering along z



$$x' = x(1 + \tau_x z / s_z),$$
$$y' = y(1 + \tau_y z / s_z), \quad z' = z.$$

Bending about z



- k_z: curvature; circular radius $1/|k_z|$.
- α_z: **azimuth** of the bending plane, not another curvature.
- Repeat for bending about x and y.

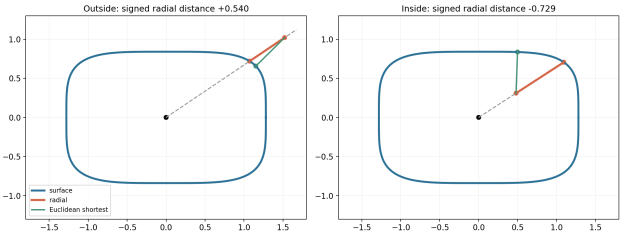
Nineteen parameters, one implicit field

Pose	Scale	Shape	Taper	Bend	**Total**
$(\mathbf{t}, R)$	$\mathbf{s}$	(ϵ_1, ϵ_2)	(τ_x, τ_y)	$\beta = (k_a, \alpha_a)_{a=x,y,z}$	
$3+3$	3	2	2	6	**19**

$$q(\mathbf{x};\Theta) = q_{\text{can}}\left(\underbrace{T^{-1} \circ B_y^{-1} \circ B_x^{-1} \circ B_z^{-1}}_{\text{undo deformations}}(R^\top(\mathbf{x}-\mathbf{t})); \mathbf{s}, \boldsymbol{\epsilon}\right)$$

- To test a world point, **undo its pose** and **undo the deformations**.
- On the surface $q(\mathbf{x};\Theta) = 1$; inside $q < 1$ (in the valid invertible regime).
- Important: T^{-1} and B_a^{-1} are **inverse geometric maps**, not inverse neural nets.

Radial distance: cheap geometry-aware supervision



Rigid case: derive it

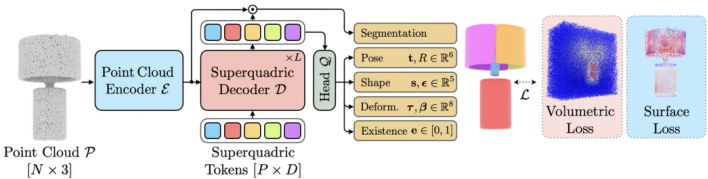
$q(\lambda \mathbf{x}) = \lambda^{2/\epsilon_1} q(\mathbf{x})$

At $q(\lambda_* \mathbf{x}) = 1$:
$\lambda_* = q(\mathbf{x})^{-\epsilon_1/2}$

$$d_{rad} = \|R^\top(\mathbf{x} - \mathbf{t})\|(1 - q(\mathbf{x}; \Theta)^{-\epsilon_1/2})$$

- $d > 0$ outside; $d < 0$ inside.
- Fast surrogate, **not Euclidean SDF**.

Inside the model: point features and primitive queries



- Input: $\mathcal{P} \in \mathbb{R}^{N \times 3}$.
- **PVCNN** point encoder: $F_{PC} \in \mathbb{R}^{N \times D}$.
- P positional **superquadric queries**: $F_{SQ} \in \mathbb{R}^{P \times D}$.
- L Transformer layers: query self-attention + point-feature cross-attention.
- Heads output $\{(\Theta_p, e_p)\}_{p=1}^{P}$ and soft point assignments.

$W = \mathrm{softmax}_p(F_{PC}\, \phi(F_{SQ})^\top) \in \mathbb{R}^{N \times P}$ Paper Fig. 3 and Eq. (4); $N = 4096$, $P = 16$, $D = 128$, $L = 3$ in experiments.

Why change the loss? Chamfer alone is insufficient



Paper Fig. 4: SuperDec vs. rigid SuperFlex vs. deformable SuperFlex.

Chamfer distance

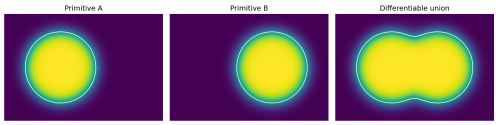
Matches samples on observed and predicted **surfaces**, not directly the **solid union**.

- Rounded-off shape boundaries.
- Gaps between neighbouring primitives.
- Poor coupling of global volume and local surface alignment.

Solution: global occupancy + local surface alignment.

New loss I: global volumetric consistency

1. Convert each radial proxy to **soft occupancy**:
   $I_p(\mathbf{x}) = \sigma\left(-\frac{d_p(\mathbf{x})}{T_{\mathrm{IoU}}}\right)$.
2. Aggregate existing primitives by a **probabilistic OR**:
   $\hat{O}(\mathbf{x}) = 1 - \prod_{p=1}^{P}(1 - e_p I_p(\mathbf{x}))$.
3. Compare with ground-truth occupancy O over uniform samples $\mathcal{V}$.



If $e_1 = e_2 = 1$:

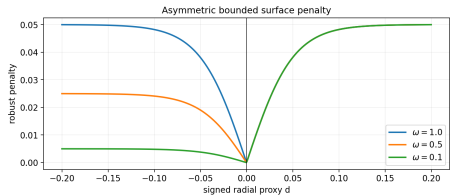
$$\hat{O} = I_1 + I_2 - I_1 I_2.$$

$$\mathcal{L}_{\mathrm{IoU}} = -\log \frac{\sum_{\mathbf{x}\in\mathcal{V}} \hat{O}(\mathbf{x}) O(\mathbf{x})}{\sum_{\mathbf{x}\in\mathcal{V}} [\hat{O}(\mathbf{x}) + O(\mathbf{x}) - \hat{O}(\mathbf{x}) O(\mathbf{x})]}$$

New loss II: surface alignment, point by point

$$\mathcal{L}_{\mathrm{SDF}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip}\, \psi\big(d(\mathbf{x}_i; \Theta_p)\big)$$

- w_{ip}: point i's soft assignment to primitive p.
- d: signed radial **surrogate**, not exact Euclidean SDF.
- ψ: robust bounded asymmetric distance penalty.



$$\psi(d) = \begin{cases} \delta \tanh(d/\delta), & d \geq 0, \\ -\omega\delta \tanh(d/\delta), & d < 0. \end{cases}$$

Compact decompositions require explicit regularization

$$\mathcal{L} = \lambda_{\text{IoU}}\mathcal{L}_{\text{IoU}} + \lambda_{\text{SDF}}\mathcal{L}_{\text{SDF}} + \lambda_o\mathcal{L}_{\text{overlap}} + \lambda_s\mathcal{L}_{\text{sparsity}} + \lambda_e\mathcal{L}_{\text{exist}}$$

Overlap

$$\mathbb{E}_{\mathbf{x}}\left[\text{ReLU}\left(\sum_p I_p(\mathbf{x}) - 1\right)\right]$$

Discourage redundant volume.

Sparsity

$$m_p = \frac{1}{N}\sum_i w_{ip}$$

Penalize excess primitive use; encourage parsimonious sets.

Existence

$$\hat{e}_p = \mathbf{1}[m_p > \epsilon_e]$$

Binary cross-entropy aligns predicted existence with usage.

Paper Eqs. (5), (10)–(11). Sparsity and existence terms are inherited from SuperDec.

Optional test-time optimization: local geometric refinement

1. **Select** active primitives: $e_p > 0.5$.
2. **Optimize** their geometry Θ_p by Adam, holding the set fixed.
3. **Replace** pointwise assignments with a global distance to the union.

$$\widetilde{d}(\mathbf{x}) = -T \log \sum_{p \in \mathcal{A}} \exp\left(-\frac{d_p(\mathbf{x})}{T}\right)$$

Smooth approximation to $\min_{p \in \mathcal{A}} d_p(\mathbf{x})$.



Paper Fig. 5. Refines existing primitives rather than training a new network.

Result: 0.72 → 0.87 IoU, but 0.0082 s → 5 s/object.

Partial scans: from self-supervision to structural completion



Paper Fig. 6: partial input; standard vs. robust model.

1. On **complete** clouds, predict and optimize high-quality primitive sets.
2. Treat the resulting sets as **pseudo-ground truth**.
3. **Fine-tune** on occluded inputs to reconstruct complete primitive structure.

Because primitives form a set, Hungarian matching removes ordering ambiguity before applying shape-based Chamfer supervision.

Experimental protocol: what was actually tested?

Object-level: ShapeNet

- 13 object categories.
- 4096 sampled points/object.
- 16 maximum primitive queries.
- 1000 training epochs.
- 4 $\times$ NVIDIA GH200 GPUs reported for training.

Partial observations

- **ASE**: quantitative evaluation with full object ground truth.
- **ScanNet++**: qualitative real-world scenes.
- Evaluate IoU, F-score, Chamfer L1/L2, #primitives, runtime.

Separate feed-forward inference from the much slower optional refinement. Paper Sec. 5.

Training-compute specification does not imply the same hardware is required for inference.

ShapeNet: the actual quantitative comparison

Method	IoU ↑	F ↑	# primitives ↓	Runtime ↓
Marching Primitives	0.56	0.18	24.10	163.41 s
SuperDec	0.59	0.28	5.79	0.0075 s
SuperFlex (rigid)	**0.70**	0.35	6.04	0.0075 s
SuperFlex (deformable)	**0.72**	**0.37**	**5.64**	0.0082 s

Largest advance

The new loss makes the **rigid** model jump from 0.59 to 0.70 IoU.

Added expressiveness

Bending + tapering improve from 0.70 to 0.72 while keeping a compact representation.

Paper Table 1, ShapeNet. Forward passes only; all numbers are paper-reported, not reproduced here.

Ablation: why do the losses work together?

Supervision	IoU ↑	L_2 ↓	Overlap ↓	# prim. ↓
SuperDec loss	0.59	0.050	4%	5.78
$\mathcal{L}_{\text{IoU}}$ only	**0.74**	0.124	32%	5.94
$\mathcal{L}_{\text{IoU}} + \mathcal{L}_{\text{SDF}}$	0.72	**0.042**	14%	5.80
$+ \mathcal{L}_{\text{overlap}}$	0.72	0.043	**8%**	**5.64**

- **IoU alone**: much stronger solid reconstruction, but too much overlap.
- **SDF term**: restores surface precision (L_2: 0.124 → 0.042).
- **Overlap term**: brings redundancy down from 14% to 8%.

Paper Table 4. Sparsity and existence are held fixed; the IoU score does not improve monotonically with every added term.

Why keep deformation and test-time optimization?

Deformation ablation (refined)

Taper/Bend	IoU	L_2
Neither	0.81	0.041
Taper only	0.83	0.038
Bend only	0.84	0.037
Both	**0.85**	**0.036**

Paper Table 5: same undeformed initializations, different allowed deformations in refinement.

Test-time optimization

	No TTO	With TTO
IoU	0.72	**0.87**
F-score	0.37	**0.49**
Runtime	**0.0082 s**	5 s

Paper Table 2: this is an accuracy–latency trade-off, not a free gain.

Refinement improves geometry; learned feed-forward inference preserves speed.

Occluded objects: generalization needs a new training signal



Paper Fig. 7: qualitative ScanNet++ scene reconstructions.

ASE partial-cloud method	IoU
SuperDec	0.20
SuperFlex, base	0.20
+ occlusion fine-tuning	0.48
+ ASE fine-tuning	**0.54**

Interpretation

Completing unseen parts requires learned priors from **complete pseudo-targets**; improving the full-cloud fit alone does not solve occlusion.

Paper Table 3, Fig. 7. ASE numbers are quantitative; ScanNet++ examples are qualitative.

Conclusions — and what remains open

Three things to remember

1. **Representation**: interpretable deformable primitives instead of unconstrained meshes.
2. **Training**: volumetric union + local surface alignment + compactness constraints.
3. **Generalization**: full-object pseudo-targets make completion from partial scans possible.

Limitations and caveats

- Approximate rather than exact Euclidean SDF.
- Optional refinement is slower.
- Dependence on generated complete-shape targets and shape priors.
- Primitive abstractions cannot express every fine detail.

How much geometry can a few meaningful parameters recover?

Primary source: Tavernini et al., ECCV 2026, arXiv:2607.01015. Code: github.com/GabrielTavernini/superflex.

Discussion / Q&A

Backup mathematics, design choices and implementation

Backup A: exact z-axis deformation equations

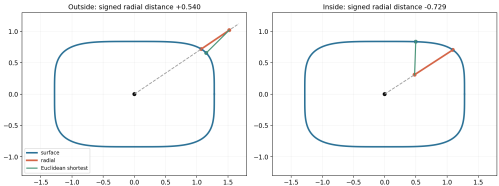
$$T^{-1}(x, y, z) = \begin{pmatrix} \frac{x}{1+\tau_x z/s_z} \\ \frac{y}{1+\tau_y z/s_z} \\ z \end{pmatrix}.$$

$$B_z^{-1}(x, y, z) = \begin{pmatrix} x - (A - r)\cos\alpha_z \\ y - (A - r)\sin\alpha_z \\ \gamma / k_z \end{pmatrix}, \quad \begin{aligned} A &= x\cos\alpha_z + y\sin\alpha_z, \\ \gamma &= \operatorname{atan2}(z, k_z^{-1} - A), \\ r &= k_z^{-1} - \sqrt{z^2 + (k_z^{-1} - A)^2}. \end{aligned}$$

Paper Appendix A. Coordinate branches and $k = 0$ limits need care; atan2 makes the intended quadrant choice explicit.

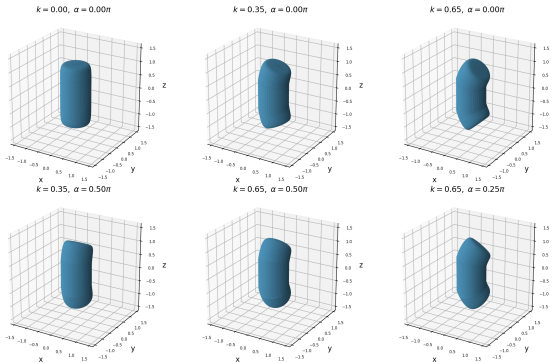
Backup B: why radial distance is exact only in a restricted sense

- Rigid superellipsoid: $q(\lambda x) = \lambda^{2/\epsilon_1} q(x)$.
- Exact ray intersection at $\lambda_* = q(x)^{-\epsilon_1/2}$; radial displacement $\|x\|(1 - \lambda_*)$.
- **Not the Euclidean SDF**: nearest surface point need not lie on the center ray.
- With nonlinear tapering/bending, $q(D^{-1}(\lambda x))$ generally is not homogeneous; substitution into the same closed form is a **surrogate even for the radial ray intersection**.



Paper Eq. (3) describes this as radial distance. Geometric qualification follows from the lack of homogeneity after deformation.

Backup C: why α is not redundant with k



Paper Appendix A.3 and Table 6.

- k: magnitude of the **curvature**.
- α: **orientation of the bending plane** within the orthogonal cross-section.
- Same k, different α: geometrically distinct deformed shapes.
- Paper ablation (subset): IoU 0.83 without α vs. 0.86 with it.

Backup D: the matching and occlusion loss

$$\mathcal{L}_{\text{geom}} = \sum_{p \in \text{matched active}} \text{CD}\Big(\mathcal{S}(\hat{\Theta}_p), \mathcal{S}(\Theta_p)\Big).$$

$$\boxed{\mathcal{L}_{\text{sup}} = \mathcal{L}_{\text{geom}} + \lambda_{\text{IoU}}\mathcal{L}_{\text{IoU}} + \mathcal{L}_{\text{rigid}}}$$

- Use a **Hungarian one-to-one assignment** between the predicted and pseudo-ground-truth primitive sets.
- Match based on **sampled surface geometry**, not raw parameter identity.
- $\mathcal{L}_{\text{rigid}}$ compares counterparts with deformations switched off: a coarse-shape prior.
- Necessary because different parameterizations, symmetries and permutations can produce similar surfaces.

Paper Sec. 4.3 and Eq. (12).

Backup E: metrics and sampling are not interchangeable

Volume IoU

$$\frac{|V_{\text{pred}} \cap V_{\text{GT}}|}{|V_{\text{pred}} \cup V_{\text{GT}}|}$$

Chamfer

Bidirectional nearest-neighbour distances between **sampled surface points**. L1 and L2 in reported tables are multiplied by 10^2.

F-score

At a paper-defined distance threshold, balance precision and recall of sampled surface correspondences.

Normal completeness

Nearest reconstructed neighbour of a GT point; average absolute unit-normal dot product. Measures alignment, not watertightness.

Paper Sec. 5; Appendix B.2. A model may improve IoU while worsening fine surface distance.

Backup F: parameter, hyperparameter and implementation details

Training choices	
Point count N	4096
Primitive queries P	16
Feature width D	128
Decoder layers L	3
Epochs	1000
Initial learning rate	3×10^{-4}
Robust fine-tuning LR	10^{-4}
IoU temperature	10^{-3}

Reproducing authors' inference

```
git clone https://github.com/
GabrielTavernini/superflex

pip install -r requirements.txt
pip install -e .
bash
scripts/download_checkpoints.sh
python demo_viser.py
```

Paper Sec. 5; official GitHub README. Inference is available with checkpoints; full training is computationally costly.

Backup G: additional 3D scene pipeline from one image



Fig. 13: SuperFlex from a single RGB Image. From an input RGB frame, we first compute instance point maps using SAM3 and MoGE. Then, we apply SuperFlex on top of each object's point map.

- SAM 3: masks instances in the original RGB image.
- MoGE: produces partial 3D point maps from a single view.
- Robust SuperFlex: turns partial object point sets into complete primitive abstractions.

Paper supplementary Fig. 13. This is an application, not the core inference pipeline or an additional benchmark result.

References and resources

- Tavernini et al., *SuperFlex: Deformable Superquadrics for Point Cloud Decomposition*, ECCV 2026, arXiv:2607.01015.
- Fedele et al., *SuperDec: 3D Scene Decomposition with Superquadric Primitives*, ICCV 2025.
- Barr, *Superquadrics and Angle-Preserving Transformations*, IEEE CG&A, 1981.
- Liu et al., *Marching-Primitives: Shape Abstraction from Signed Distance Function*, CVPR 2023.

- SuperFlex project: `https://superflex3d.github.io/`
- Source and checkpoints: `https://github.com/GabrielTavernini/superflex`

All source-paper figures included for scholarly presentation and credited on their slides. Notebook illustrations are original.